

Assignment 2 (Weeks 3 and 4)

- **Part I: Short Answer / Reasoning Questions**

Q1. Consider a Student table where the phone_number column allows NULL. A classmate writes the following query to count students without a phone number: `SELECT COUNT(*) FROM Student WHERE phone_number = NULL`; This query always returns 0, even when such students exist. Explain why, and rewrite the query correctly. Then explain, in general terms, why NULL is excluded from most aggregate computations and comparison operations.

-> In SQL, NULL represents an absent or unknown value, not a blank or zero value.

Because it is unknown, any direct comparison using `=` evaluates to unknown instead of True.

`SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;`

NULL is excluded from most calculations because you cannot perform reliable arithmetic on unknown data.

Q2. A query uses `SELECT DISTINCT department, COUNT(*) FROM Employee GROUP BY department`; A student argues the DISTINCT keyword here is redundant. Is the student correct? Justify your answer by explaining what DISTINCT would actually be doing in this specific query.

-> yes, the `GROUP BY department` clause already create the dataset into unique rows per department. Adding DISTINCT does not change the result because there are no duplicate rows remaining after grouping.

Q3. You run two queries on the same Customer and Order tables, one using INNER JOIN and one using LEFT JOIN (Customer as the left table) and get a different row count. What does this difference in row count tell you about the data, without looking at the tables directly? Describe the scenario that would cause the LEFT JOIN to return more rows.

-> This tells you that there are customers in your database who have never placed an order.

An INNER JOIN only outputs rows where a match exists on both sides. If you have 100 total customers, but only 80 have active orders, the INNER JOIN yields 80 rows. A Left Join preserves all 100 rows from the left table (Customer), populating the missing order

information with NULL for the remaining 20 customers.

Q4. Explain why the following query is invalid in standard SQL, and correct it: `SELECT department, employee_name, AVG(salary) FROM Employee GROUP BY department`; In your explanation, address what it would conceptually mean for SQL to "allow" this query to run as-is i.e., what value would it even display for `employee_name`?

-> You are selecting `employee_name` without including it in the `GROUP BY` clause or wrapping it in an aggregate function like `MAX()` or `MIN()`

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Q5. A manager wants the list of departments where the average salary exceeds ₹80,000. A student writes: `SELECT department, AVG(salary) FROM Employee WHERE AVG(salary) > 80000 GROUP BY department`; This fails. Identify the error, explain why `WHERE` cannot be used here (in terms of query execution order), and provide the corrected query.

-> the aggregate function `AVG(salary)` is placed inside the `WHERE` Clause.

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. For each of the following, state whether a correlated or non-correlated subquery is more appropriate, and briefly justify:

- (a) Find all products priced above the overall average product price.
- (b) Find all employees who earn more than the average salary of their own department. What is the key performance implication of a correlated subquery compared to a non-correlated one when the outer table is large?

-> (a) `SELECT *`
`FROM Product`

```

WHERE price >
(
    SELECT AVG(price)
    FROM Product
);

```

```

(b) SELECT *
    FROM Employee e
    WHERE salary >
    (
        SELECT AVG(salary)
        FROM Employee
        WHERE department = e.department
    );

```

The department average differs for each employee. The inner query must use the current employee's department.

Part II: Long Answer Question

Scenario

Reuse the online food delivery platform schema from Assignment 1:

Customer(customer_id, name, email, phone)

Restaurant(restaurant_id, name, cuisine_type, rating)

Menuitem(item_id, restaurant_id, item_name, price)

Order(order_id, customer_id, restaurant_id, order_date, status)

OrderItem(order_id, item_id, quantity)

For each sub-question below, write the SQL query and explain, in one or two sentences, why you chose the specific construct you used (join type, subquery vs. join, presence/absence of HAVING, etc.).

(a) List all customers who have never placed an order. (Think carefully about which join type or subquery form correctly captures "never", a naive INNER JOIN will not work.)

-> A left join record all the customers and with no order get NULL value in order column.

Tracking the Null identifies all the customer who never placed a order.

```

SELECT customer_id,

```

```

        name,
        email,
        phone
FROM Customer
LEFT JOIN Orders
ON Customer.customer_id = order.customer_id
WHERE order.order_id IS NULL;

```

(b) For each restaurant, find the average order value, where an order's value is the sum of price × quantity across its items. Only include restaurants with more than 5 orders.
 -> Compute each order using SUM(price X quantity).then group restaurant and calculate the average.HAVING is used because the condition depends on grouped results.

```

SELECT restaurant_id,
       AVG(order_value) AS avg_order_value
FROM
(
  SELECT o.order_id,
         o.restaurant_id,
         SUM(m.price * oi.quantity) AS order_value
  FROM Orders o
  JOIN OrderItem oi
  ON o.order_id = oi.order_id
  JOIN MenuItem m
  ON oi.item_id = m.item_id
  GROUP BY o.order_id, o.restaurant_id
) AS OrderValues
GROUP BY restaurant_id
HAVING COUNT(*) > 5;

```

(c) Find restaurants whose average order value (as computed in (b)) is higher than the platform-wide average order value. (This requires nesting your answer to (b) inside another query think about whether you need a correlated subquery here, and explain why or why not.)
 -> A non-correlated subquery is sufficient because platform-wide average order value is calculated once and compared with each restaurant's average.

```

SELECT restaurant_id,
       AVG(order_value) AS avg_order_value
FROM
(
  SELECT o.order_id,

```

```

        o.restaurant_id,
        SUM(m.price * oi.quantity) AS order_value
FROM Orders o
JOIN OrderItem oi
    ON o.order_id = oi.order_id
JOIN MenuItem m
    ON oi.item_id = m.item_id
GROUP BY o.order_id, o.restaurant_id
) AS OrderValues
GROUP BY restaurant_id
HAVING AVG(order_value) >
(
    SELECT AVG(order_value)
    FROM
    (
        SELECT o.order_id,
            SUM(m.price * oi.quantity) AS order_value
        FROM Orders o
        JOIN OrderItem oi
            ON o.order_id = oi.order_id
        JOIN MenuItem m
            ON oi.item_id = m.item_id
        GROUP BY o.order_id
    ) AS PlatformOrders
);

```

(d) Find the most ordered menu item (by total quantity) for each restaurant. Briefly describe one limitation of basic GROUP BY + MAX approaches for "top-N per group" problems, even if your query handles this particular case correctly.

-> GROUP BY + MAX works for finding maximum values, but it becomes difficult when multiple items tie for first place or when we need the top 3/top N items. Window functions such as ROW_NUMBER() are usually better for top-N problems.

```

SELECT r.restaurant_id,
       m.item_id,
       m.item_name,

```

```

        SUM(oi.quantity) AS total_quantity
FROM Restaurant r
JOIN MenuItem m
    ON r.restaurant_id = m.restaurant_id
JOIN OrderItem oi
    ON m.item_id = oi.item_id
GROUP BY r.restaurant_id,
        m.item_id,
        m.item_name
HAVING SUM(oi.quantity) =
(
    SELECT MAX(item_total)
    FROM
    (
        SELECT SUM(oi2.quantity) AS item_total
        FROM MenuItem m2
        JOIN OrderItem oi2
            ON m2.item_id = oi2.item_id
        WHERE m2.restaurant_id = r.restaurant_id
        GROUP BY m2.item_id
    ) AS X
);

```

Part III: Normalization

Scenario

A junior developer has designed the following single table to store course enrollment data:

Enrollment(student_id, student_name, course_id, course_name, instructor_id, instructor_name, instructor_office, grade, enrollment_date)

Assume the following holds in this system:

- A course has exactly one instructor.
- An instructor has exactly one office.
- A student can enroll in multiple courses, and a course has multiple students.
- (student_id, course_id) together uniquely identify a grade and enrollment date.

Q1. List the functional dependencies implied by the scenario above.

-> $\text{student_id} \rightarrow \text{student_name}, \text{course_name}$
 $\text{course_id} \rightarrow \text{course_name}, \text{course_instructor}$
 $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office},$
 $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}$
 $(\text{student_id}, \text{course_id}) \rightarrow \text{enrollment_date}$

Q2. Is this table in 1NF? Justify your answer.

-> Every attribute contains one values and there are no repeating groups or multivalued attributes. Therefore the table satisfies First Normal Form (1NF).

Q3. Identify the partial dependency that violates 2NF, and explain why it is a problem in terms of update/insertion/deletion anomalies (give one concrete example of an anomaly that could occur).

-> $(\text{student_id}, \text{course_id})$ candidate key

$\text{student_id} \rightarrow \text{student_name}$

$\text{course_id} \rightarrow \text{course_name}$

$\text{course_id} \rightarrow \text{instructor_id}$

Why is this a problem?

Update Anomaly-> Suppose course name "DBMS" changes to "Advanced DBMS". It must be updated in every enrollment record. If one row is missed, inconsistent data appears.

Insertion Anomaly-> Cannot add a new course until at least one student enrolls.

Deletion Anomaly-> Deleting the last student enrolled in a course removes course information completely.

Q4. Identify the transitive dependency that violates 3NF, and explain the anomaly it causes.

->

Transitive Dependency:
 $\text{course_id} \rightarrow \text{instructor_id}$

$\text{instructor_id} \rightarrow \text{instructor_name}$
 $\text{instructor_id} \rightarrow \text{instructor_office}$

If an instructor changes office number, every enrollment row for courses taught by that instructor must be updated.

Missing one row causes inconsistent office information.

Q5. Decompose the table into a set of relations that are in 3NF. Use the notation $\text{TableName}(\text{col1}, \text{col2}, \text{col3})$ and underline (or otherwise indicate) each primary key. For each resulting table, briefly state which functional dependency justified separating it out.

->

Student

Student(
 $\underline{\text{student_id}}$ PK,
 student_name
)

Dependency:

$\text{student_id} \rightarrow \text{student_name}$

Instructor

Instructor(
 $\underline{\text{instructor_id}}$ PK,
 instructor_name,
 instructor_office
)

Dependency:

$\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$

Course

Course(
 $\underline{\text{course_id}}$ PK,
 section_id PK,
 course_name,
 credits
)

course_id PK,

course_name,

instructor_id

)

Dependency:

course_id → course_name, instructor_id

Enrollment

Enrollment(

student_id PK/FK,

course_id PK/FK,

grade,

enrollment_date

)

Dependency:

(student_id, course_id) → grade, enrollment_date

Part IV: Design Justification (150–200 words)

Reflecting on Part II, address the following:

- For one query in Part II, explain a case where you could have written it using either a JOIN or a subquery, but chose one over the other. What factors (readability, correctness for NULL/duplicate handling, or performance) drove your choice?
- In Part III, explain why normalizing the Enrollment table trades off against query convenience i.e., what becomes harder to do once the table is split into 3NF form, and why might a real system sometimes intentionally keep some denormalization?

->

One query from Part II that could be written using either a JOIN or a subquery is the query for customers who never placed an order. I chose a LEFT JOIN with a NULL check because it clearly expresses the requirement of keeping all customers and then identifying those without matching orders. An alternative solution using NOT EXISTS is also valid. The LEFT JOIN approach is easier to understand for many readers, while NOT EXISTS may perform better in some database systems. The choice depends on readability, optimizer behavior, and how NULL values are handled.

Normalization in Part III improves data consistency by removing redundancy. However, after decomposing the Enrollment table into Student, Course, Instructor, and Enrollment tables, retrieving complete enrollment information requires multiple joins. Queries become longer and sometimes slower because data is spread across several tables. In real systems, some denormalization may be intentionally introduced to improve reporting performance or reduce the number of joins required for frequently executed queries. Thus, normalization improves integrity, while denormalization can improve convenience and performance when carefully controlled.